DJ-01 · SQUAD DJOLOFF

Modèle de données Learner Success

Les six tables du système, leurs types, leurs relations — et les trois décisions de conception qui les expliquent.

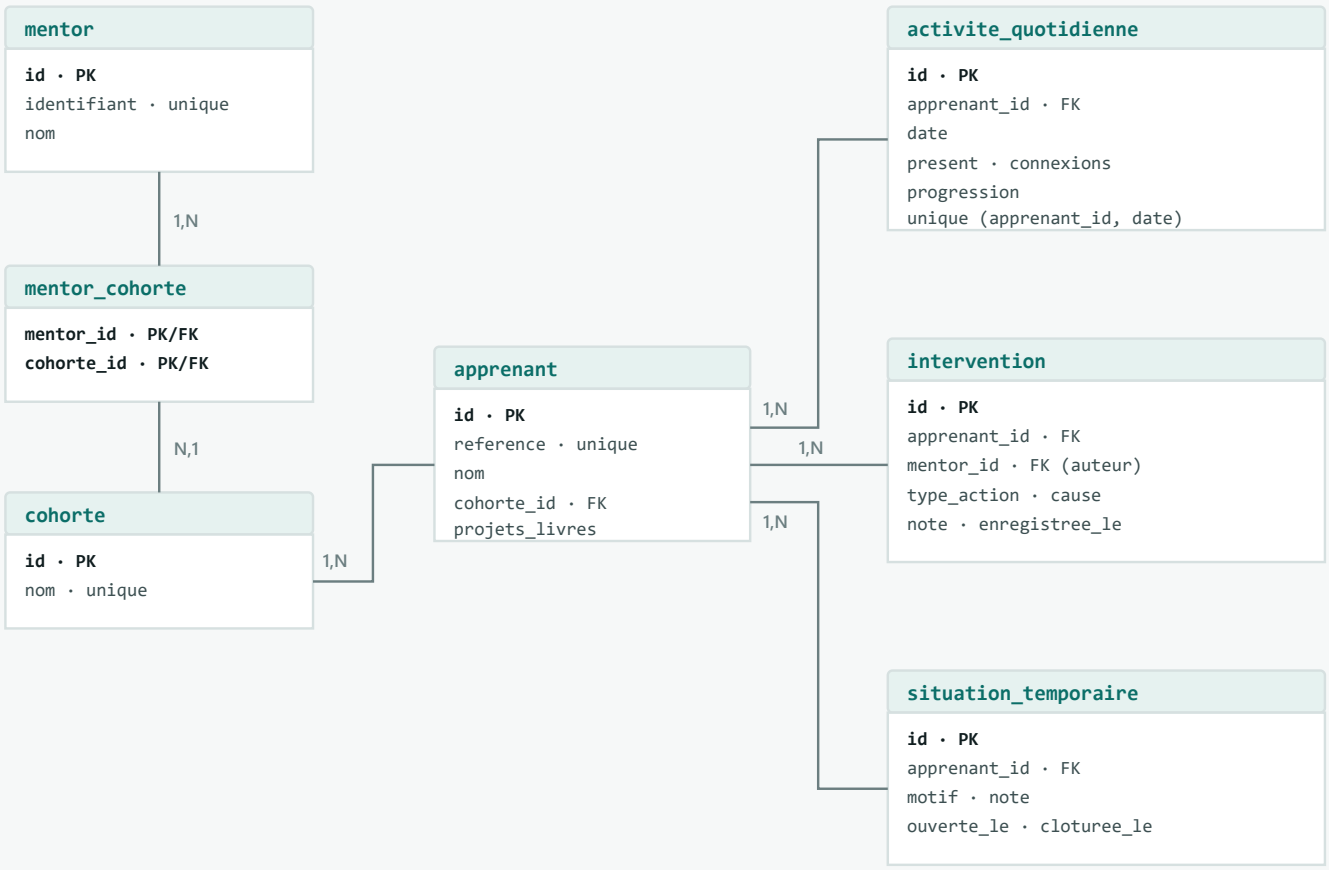
D'où l'on part

Le MVP fonctionne **sans base de données** : les données d'activité sont lues depuis des fichiers CSV, le suivi mentor est écrit en JSON. Ce choix a permis de livrer vite, et il tient parce qu'un seul fichier lit les données — tout le reste travaille sur le résultat.

Ce document décrit le **modèle cible** : ce que deviennent ces fichiers en base relationnelle, sans rien changer aux règles métier.

AUJOURD'HUI	DEMAIN
`apprenants.csv`	tables `apprenant` + `cohorte`
`historique.csv`	table `activite_quotidienne`
`mentors.csv`	tables `mentor` + `mentor_cohorte`
`interventions.json`	table `intervention`
`situations.json`	table `situation_temporaire`

Le schéma



Un mentor suit plusieurs cohortes, une cohorte réunit plusieurs apprenants, chaque apprenant porte son activité et son suivi.

L'auteur d'une intervention est un **mentor** : la table `intervention` porte une clé étrangère vers `mentor`, non représentée par un trait pour ne pas surcharger le schéma.

Les tables

`cohorte`

COLONNE	TYPE	CONTRAINTE
`id`	SERIAL	`PK`
`nom`	VARCHAR(120)	NOT NULL · UNIQUE

mentor

COLONNE	TYPE	CONTRAINTE
id	SERIAL	PK
identifiant	VARCHAR(60)	NOT NULL · UNIQUE — ex. papa.malick
nom	VARCHAR(120)	NOT NULL

mentor_cohorte

Table d'association : un mentor suit plusieurs cohortes, une cohorte peut être suivie par plusieurs mentors.

COLONNE	TYPE	CONTRAINTE
mentor_id	INTEGER	PK FK → mentor
cohorte_id	INTEGER	PK FK → cohorte

apprenant

COLONNE	TYPE	CONTRAINTE
id	SERIAL	PK
reference	VARCHAR(20)	NOT NULL · UNIQUE — ex. APP001
nom	VARCHAR(120)	NOT NULL
cohorte_id	INTEGER	NOT NULL FK → cohorte
projets_livres	SMALLINT	NOT NULL · ≥ 0

activite_quotidienne

COLONNE	TYPE	CONTRAINTE
id	BIGSERIAL	PK
apprenant_id	INTEGER	NOT NULL FK → apprenant
date	DATE	NOT NULL
present	BOOLEAN	NOT NULL
connexions	SMALLINT	NOT NULL · ≥ 0
progression	NUMERIC(5,2)	NOT NULL · 0 à 100

UNIQUE (apprenant_id, date) — une seule ligne par apprenant et par jour

intervention

COLONNE	TYPE	CONTRAINTE
id	BIGSERIAL	PK
apprenant_id	INTEGER	NOT NULL FK → apprenant
mentor_id	INTEGER	NOT NULL FK → mentor — l'auteur
type_action	ENUM	NOT NULL · appel · entretien · autre
cause	ENUM	NULL autorisé · 6 valeurs
note	VARCHAR(500)	NULL autorisé
enregistree_le	TIMESTAMPTZ	NOT NULL · DEFAULT now()

situation_temporaire

COLONNE	TYPE	CONTRAINTE
id	BIGSERIAL	PK
apprenant_id	INTEGER	NOT NULL FK → apprenant
motif	VARCHAR(120)	NOT NULL · 2 caractères minimum
note	VARCHAR(500)	NULL autorisé
ouverte_le	TIMESTAMPTZ	NOT NULL · DEFAULT now()
cloturee_le	TIMESTAMPTZ	NULL = situation en cours

Les relations

ENTRE	CARDINALITÉ	LECTURE
mentor ↔ cohorte	N,N	Un mentor suit plusieurs cohortes ; une cohorte peut avoir plusieurs mentors
cohorte → apprenant	1,N	Une cohorte réunit plusieurs apprenants ; un apprenant appartient à une seule
apprenant → activite	1,N	Un apprenant a une ligne d'activité par jour observé
apprenant → intervention	1,N	Un apprenant peut recevoir plusieurs interventions
mentor → intervention	1,N	Un mentor est l'auteur de plusieurs interventions
apprenant → situation	1,N	Plusieurs situations dans le temps, **une seule ouverte à la fois**

Trois décisions de conception

1 · Le niveau de risque n'est pas stocké

Il n'existe **aucune colonne** `score` ou `niveau_risque`. Le feu rouge / orange / vert est recalculé à chaque consultation, à partir des règles et des données du moment.

Pourquoi : un score stocké se périme dès que l'activité change, et devient modifiable à la main — donc contestable. Recalculé, il est toujours juste et toujours justifiable.

2 · Les totaux sur 30 jours sont dérivés, pas dupliqués

Le nombre d'absences, le total des connexions, la progression et la date de dernière connexion **ne sont pas des colonnes** de `apprenant` : ils se calculent depuis `activite_quotidienne`.

```
-- Les indicateurs du scoring, calcules depuis l'activite
CREATE VIEW apprenant_indicateurs AS
SELECT
    a.id,
    COUNT(*) FILTER (WHERE NOT q.present)      AS nb_absences,
    COALESCE(SUM(q.connexions), 0)               AS nb_connexions,
    MAX(q.progression)                           AS progression,
    MAX(q.date) FILTER (WHERE q.connexions > 0)  AS derniere_connexion
FROM apprenant a
LEFT JOIN activite_quotidienne q
       ON q.apprenant_id = a.id
      AND q.date > CURRENT_DATE - INTERVAL '30 days'
GROUP BY a.id;
```

Pourquoi : une donnée dupliquée finit toujours par diverger de sa source. Ici l'activité quotidienne est la seule vérité, le reste en découle.

3 · Une seule situation temporaire ouverte à la fois

Garanti par la base elle-même, pas seulement par le code :

```
CREATE UNIQUE INDEX situation_ouverte_unique
    ON situation_temporaire (apprenant_id)
 WHERE cloturee_le IS NULL;
```

Pourquoi : deux pauses simultanées pour le même apprenant n'ont pas de sens, et rendraient la clôture ambiguë. Un index unique partiel rend la situation impossible, même si le code se trompe.

> **À RETENIR**
>
> La situation temporaire **n'altère jamais le niveau de risque** — aucune colonne ne les relie. Elle l'explique, elle ne le masque pas : l'apprenant reste rouge, mais le mentor comprend pourquoi.

Le schéma en SQL

```sql
CREATE TYPE type_action AS ENUM ('appel', 'entretien', 'autre');
CREATE TYPE cause_desengagement AS ENUM (
    'financiere', 'technique', 'personnelle',
    'familiale', 'motivationnelle', 'autre');

CREATE TABLE cohorte (
    id    SERIAL PRIMARY KEY,
    nom   VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE mentor (
    id           SERIAL PRIMARY KEY,
    identifiant  VARCHAR(60)  NOT NULL UNIQUE,
    nom          VARCHAR(120) NOT NULL
);

CREATE TABLE mentor_cohorte (
    mentor_id   INTEGER NOT NULL REFERENCES mentor(id)  ON DELETE CASCADE,
    cohorte_id  INTEGER NOT NULL REFERENCES cohorte(id) ON DELETE CASCADE,
    PRIMARY KEY (mentor_id, cohorte_id)
);

CREATE TABLE apprenant (
    id              SERIAL PRIMARY KEY,
    reference       VARCHAR(20)  NOT NULL UNIQUE,
    nom             VARCHAR(120) NOT NULL,
    cohorte_id      INTEGER      NOT NULL REFERENCES cohorte(id),
    projets_livres  SMALLINT     NOT NULL DEFAULT 0 CHECK (projets_livres >= 0)
);

CREATE TABLE activite_quotidienne (
    id            BIGSERIAL PRIMARY KEY,
    apprenant_id  INTEGER       NOT NULL REFERENCES apprenant(id) ON DELETE CASCADE,
    date          DATE          NOT NULL,
    present       BOOLEAN       NOT NULL,
    connexions    SMALLINT      NOT NULL DEFAULT 0 CHECK (connexions >= 0),
    progression   NUMERIC(5,2)  NOT NULL CHECK (progression BETWEEN 0 AND 100),
    UNIQUE (apprenant_id, date)
);

CREATE TABLE intervention (
    id              BIGSERIAL PRIMARY KEY,
    apprenant_id    INTEGER      NOT NULL REFERENCES apprenant(id) ON DELETE CASCADE,
    mentor_id       INTEGER      NOT NULL REFERENCES mentor(id),
    type_action     type_action  NOT NULL,
    cause           cause_desengagement,
    note            VARCHAR(500),
    enregistree_le  TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE situation_temporaire (
    id            BIGSERIAL PRIMARY KEY,
    apprenant_id  INTEGER      NOT NULL REFERENCES apprenant(id) ON DELETE CASCADE,
    motif         VARCHAR(120) NOT NULL CHECK (LENGTH(motif) >= 2),
    note          VARCHAR(500),
    ouverte_le    TIMESTAMPTZ  NOT NULL DEFAULT now(),
```

```
    cloturee_le   TIMESTAMPTZ
);

-- Une seule situation ouverte par apprenant
CREATE UNIQUE INDEX situation_ouverte_unique
    ON situation_temporaire (apprenant_id) WHERE cloturee_le IS NULL;

-- Index de lecture : les deux requetes les plus frequentes
CREATE INDEX ON activite_quotidienne (apprenant_id, date DESC);
CREATE INDEX ON intervention (apprenant_id, enregistree_le DESC);
```

Ce que la migration changerait dans le code

Peu de choses, et c'est voulu. La lecture des données est isolée dans une seule couche.

FICHIER	IMPACT
`model/loader.py`	**Remplacé** — requêtes SQL au lieu de la lecture CSV
`model/historique.py`	**Remplacé** — même chose pour l'activité
`model/suivi.py`	**Remplacé** — INSERT/UPDATE au lieu de l'écriture JSON
`model/mentors.py`	**Remplacé** — jointure sur mentor_cohorte
`model/scoring.py`	Inchangé
`model/reengagement.py`	Inchangé
`api/`	Inchangé
`frontend/`	Inchangé

Les règles métier, l'API et l'interface ne bougent pas. **Seule la source des données change.**